

Compte-rendu TP7

B - Contrôleur DMA

B1

- **SOURCE:** Accès en écriture ignoré si l'automate n'est pas en IDLE.
- **DEST:** Accès en écriture ignoré si l'automate n'est pas en IDLE.
- **LENGTH/STATUS:** Un accès en écriture initie le transfert du DMA suivant les registres SOURCE et DEST.
Un accès en lecture donnera l'état actuel de l'automate du DMA.
- **RESET:** Accès en écriture arrête le transfert actuel du DMA et force l'automate à l'état IDLE. Cela permet également de reconnaître une IRQ.
- **IRQ_DISABLED:** Si ce registre contient 1, les interruptions ne sont ignorées sur le DMA.

L'adresse de base du composant doit être alignée sur une frontière de bloc de 32 octet car seuls les 5 bits de poids faibles sont décodés et permettent d'adresser donc plus facilement les registres du DMA. Comme il y a 5 registres de 4 octets, on a 20 octet, en arrondissant à la puissance de 2 supérieur on arrive donc à 32 octets.

B2

L'argument burst permet de spécifier le nombre maximum de mots que l'on veut transférer en 1 transfert en rafale.

B3

Pour contrôler le coprocesseur DMA, il faut deux automates. L'un pour gérer les transferts en rafales et les accès au bus, l'autre pour gérer l'état des commandes du processeur. Ceci permet de séparer deux tâches spécifiques indépendantes. L'automate Target indiquera donc au processeur si le DMA est capable de traiter sa requête ou s'il est occupé par exemple.

B4

La bascule *r_stop* a pour fonction de signaler à l'automate Master d'arrêter son transfert s'il n'est pas en train de faire un transfert en rafale. Cela permet donc de synchroniser l'automate Master avec l'automate Target lorsque ce dernier

La bascule *r_stop* permet de synchroniser les deux automates Target et Master.

Target peut dire à Master de ne pas faire de transfert, dans ce cas la bascule *r_stop* est mise à 1 par Target. Cela permet également d'interrompre Master s'il est en train de faire un transfert. Mais un transfert en rafale ne peut être interrompu.

Sinon Target peut dire à Master de faire un transfert en mettant *r_stop* à 0.

B5



La longueur d'une rafale en mots de 32 bits est de 16.

L'avantage d'utiliser des grosses rafales est le nombre de cycles économisés par la réduction du nombre de requête au bus ainsi que les cycles servant à initier la transaction et la terminer.\

C2

L'adresse de base du composant DMA est 0x93000000.

Le numéro de cible du DMA pour le BCU est 6:

```
#define DMA_INDEX 6
```

Le numéro de maître du DMA pour le BCU est 1:

```
bcu.p_req[nprocs] (signal_req_dma);  
bcu.p_gnt[nprocs] (signal_gnt_dma);
```

Le port d'entrée du composant ICU connecté au DMA est le port 0:

```
IRQ_IN[0] : DMA
```

D - Application logicielle

D1

Le composant qui effectue le transfert de pixel est le processeur lui-même. L'appel car il utilise la fonction memcpy définit dans *common.h*.

La fonction est donc bloquante car c'est une boucle while exécutée par le processeur.

D2

Avec l'appel memcpy:

- Temps de construction de l'image: 2 432 848 cycles
- Temps d'affichage de l'image: 2 870 879 - 2 432 848 = 438 031 cycles

D3

fb_sync_write fait appel à la fonction memcpy, qui s'exécute sur le processeur, pour copier une zone mémoire utilisateur vers une destination donnée.

fb_write configure les registre du composant DMA pour qu'il transfère la mémoire à la manière de memcpy mais avec des transactions en rafales et en parallèle du processeur.

L'appel à *fb_completed()* permet de vérifier si le DMA est prêt à faire une nouvelle transaction, cela permet donc de ne pas effacer le buffer du DMA d'une transaction en cours.

D4

Avec le DMA on obtient cette-fois ci:

- Temps de construction de l'image: 2 432 848 cycles

- Temps d'affichage de l'image: $2\,476\,462 - 2\,432\,848 = 43\,614$ cycles, soit 10 fois moins de temps d'affichage qu'avec memcpy!

D5

Sans l'appel à `fb_completed()` et 1 mot par rafale:

- Temps de construction de l'image: 2 432 844 cycles
- Temps d'affichage de l'image: $2\,436\,605 - 2\,432\,844 = 3\,761$ cycles soit 1000 fois moins qu'en attendant avec `fb_completed`!

Le défaut est que lors de l'affichage des différentes étapes, sur le bords gauche de l'image, les carrés ne sont pas de la même taille que sur le reste de l'image.

Ceci est dû au fait que le DMA est probablement trop lent par rapport au programme utilisateur qui n'attend plus que le DMA ait fini d'écrire dans le frame buffer avec l'appel à `fb_completed()`. Les images se mélangent donc car le programme demande au DMA d'écrire plusieurs images en même temps.

D6

La variable `_dma_busy` est mise à 1 dans la fonction `_fb_read` et `_fb_write`. Elle est mise à 0 dans la fonction `_isr_dma`.

E - Pipeline logiciel

E1

Pour passer de la période (n) à la période ($n+1$) il faut vérifier que la constructions et l'affichage soit finit dans les deux buffers pour ne pas avoir d'accès concurrents. Comme l'affichage est beaucoup plus rapide que la construction on doit vérifier seulement que la constructions est finie.

E2

- Temps de construction de l'image: 3 104 741 cycles
- Temps d'affichage de l'image: $6\,218\,657 - 6\,215\,105 = 3\,552$ cycles

On a un temps d'affichage un peu moins long de 2 000 cycles environ.

Le gain n'est pas très significatif car l'affichage est beaucoup plus court que la construction de l'image d'environ un facteur 1 000.

La parallélisation de ces deux tâches ne provoque donc pas un gain significatif.

12 345 946 cycles DMA

15 606 323 cycles PIPE

F - Traitement d'erreurs

F1

Le programme utilisateur ne doit pas pouvoir accéder directement à la zone protégée de l'espace adressable, `fb_write` vérifie donc que `fb_write` n'écrive pas directement le contenu de l'espace protégé car l'utilisateur n'en

a pas le droit. Ce type d'erreur doit absolument être détecté par le contrôleur DMA car l'utilisateur pourrait par exemple modifier le registre busy du DMA ce qui n'est pas prévu par le comportement du composant et pourrait provoquer des bugs.

F2

exit(1) (from pibus_dma.cpp)
 SYSCALL_EXIT (from stdio.c)
 _exit() (from common.c)

Dans *pibus_dma.cpp*, le contrôleur émet une requête d'irq à la fin de la transition lors d'une erreur:

```
if (((r_master_fsm == DMA_SUCCESS) ||
    (r_master_fsm == DMA_READ_ERROR) ||
    (r_master_fsm == DMA_WRITE_ERROR)) && (r_irq_disable == 0)) {
    p_irq = true;
}
```

Ceci émet donc le signal *signal_irq_dma* qui connecte le DMA et l'ICU dans *tp5_top.cpp*:

```
dma.p_irq (signal_irq_dma); icu.p_irq_in0;
```

L'ICU va donc rediriger cette interruption vers les processeurs concernés par le mask.

Ensuite cette interruption va provoquer l'appel à *_giet* qui va appeler *_int_handler* puis *_int_demux*.

Cela va appeler *_isr_dma* qui (dans *irq_handler.c*) qui correspond à la routine d'interruption du DMA.

L'isr va enregistrer dans le status du DMA dans *DMA_LEN*, afin de savoir si la transaction s'est terminée (=0) ou a été interrompue (>0).

L'isr va également mettre à 0 la variable *_dma_busy* afin de déclarer la fin de la transaction du DMA.

L'appel à la fonction *fb_completed()* (*drivers.c*) permet donc d'attendre que la transaction soit finie, puis de vérifier s'il elle s'est faite jusqu'au bout (*DMA_LEN* = 0) ou si elle n'a pas pu se terminer correctement (*DMA_LEN* > 0). Cet appel permet donc de constater si le DMA a reçu un erreur du Bus.

F3

- Si l'adresse correspond à une adresse non définie comme 0x0 ou 0x7, l'appel à *fb_completed()* retourne la valeur 1. Cela signifie donc que *isr_dma* a été appelée alors que la transaction n'était pas finie car *DMA_LEN* n'est pas égal à 0 vu que *fb_completed* a renvoyé 1.
- Si l'adresse appartient à la zone protégée > 0x80000000, c'est le driver du *Frame_buffer* avec l'appel à *fb_write* qui échoue.

G - Amélioration du parallélisme

- Temps de construction de l'image: 794 765 cycles, soit 3 fois plus rapide qu'avec le DMA et un processeur!
- Temps d'affichage de l'image: 4 006 369 - 4 002 559 = 3 810 cycles, il reste constant car le calcul est toujours effectué en parallèle par le DMA que l'on doit attendre avant le début de chaque itération.

- Temps total: 4 062 160 cycles, soit 4 fois plus rapide qu'avec le DMA et un seul processeur! Ceci est cohérent avec le fait que l'on ait découpé et parallélisé la construction de l'image en 4.

Les problèmes rencontrés sont les suivants:\

- On change le nombre de processeurs dans *config.h*.\
- Lorsque l'on charge une adresse il faut utiliser la.\
- Pour définir le numéro du processeur grâce à son id, on fait un "and" entre son id et 0b11 pour - retenir que les MSB.\
- Pour chaque processeur on calcule bien l'offset correspondant dans la pile par rapport au processeur d'ID inférieur.\
- Il faut activer SNOOP avec -SNOOP 1.\
- Chaque processeur saute au même main, il faut donc toujours sauter à 0(seg_data_base).\
- Une barrière a un index, chaque processeur utilisant une même barrière utilisent le même index. La barrière est initialisée une seule fois par 1 processeur.\
- En déclarant les buffer en dehors du main, ils deviennent des variables globales qui ne sont pas situées sur la pile. Ils sont donc partagés par tous les processeurs.\
- Il faut activer le mask du DMA que pour le processeur 0, et faire l'affichage uniquement avec le processeur 0!